DAILY JAVA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

# Java – Beautifully Broken

## Apache Camel fixes 39 vulnerabilities in a week, and Shenandoah learns to wait instead of stopping the world

Issue #15 | Tue, 20 Oct 2026 | ⏱️ ~40 min read | Level 3/5

☕ **Today's two stories**

1. **Apache Camel just published 39 security advisories in five days.** Twenty-eight of them came from an AI scanner run under Anthropic's Project Glasswing, and fourteen are rated HIGH, including two remote-code-execution paths through Java deserialization. We read every advisory: here's what Camel is, the six kinds of mistake these bugs fall into, which versions fix them, and what to strip, filter or upgrade this week if Camel is anywhere in your stack.
2. **Shenandoah no longer stops the world when it falls behind.** On 19 October, a 112-file commit removed Shenandoah's "degenerated" garbage collection from JDK 28. When the collector can't keep up, the thread that needs memory now waits while the concurrent cycle finishes, and the collector adds worker threads instead. We explain the old fallback (and measure it on JDK 21), the new one, and what changes in your logs and dashboards.
3. **On the radar:** 7 more things that happened, listed at the end.

## STORY 1 Apache Camel's 39-CVE week

APACHE CAMEL · SECURITY · ADVISORIES PUBLISHED 15–19 OCT 2026 · ~16 MIN READ

Most projects publish a handful of security advisories a year. Apache Camel published 38 on a single Thursday, and a 39th the following Monday. That isn't because Camel suddenly got worse. It's because, for the first time, somebody pointed an AI model at the whole codebase and kept going until the findings ran out. If your company moves files, messages or HTTP calls between systems, there's a fair chance Camel is somewhere in the path, possibly inside a Spring Boot app nobody has touched in a year. Here's what was found, why these bugs keep appearing in integration code, and exactly what to do about it.

### WHAT HAPPENED

On 15 October 2026 the Apache Camel project published **38 security advisories** at once. In a blog post the next day, Camel committer Andrea Cosentino summarised them: thirteen are rated HIGH and twenty-five MEDIUM. "Twenty-eight come from findings made by Claude as part of the Glasswing initiative at the Apache Software Foundation, and ten were reported by security researchers."[1] The 28 AI findings are credited in their advisories to the "Claude Security Scan Tool".[1] On Monday, 19 October, a 39th advisory followed: **CVE-2026-88900**, a HIGH-severity flaw in the JGroups component that "can lead to remote code execution", reported by a researcher credited as h1ei1.[2,3] InfoQ's weekly Java roundup covered the batch the same day.[4]

All 39 are fixed in three releases: **Camel 4.23.0** (15 October), **4.22.1** (17 September) and **4.18.5** (11 October).[1,5] Notice the dates: 4.22.1 shipped a month before the advisories went public. That's standard practice at the Apache Software Foundation: fix quietly, release, then disclose, so attackers don't get a head start. It also means that if you upgraded to 4.22.1 in September, you were already protected without knowing it.

The project's upgrade advice is short:[1]

| You are on | Upgrade to |
|---|---|
| 4.22.x (LTS) | 4.22.1 |
| 4.18.x (LTS) | 4.18.5 |
| Any other release | 4.23.0, or 4.22.1 / 4.18.5 to stay on an LTS line |

The uncomfortable line is the one underneath: "The 4.14.x LTS line reached end of life after 4.14.9 and does not receive these fixes. 35 of the 38 advisories affect 4.14.x."[1] The 4.14 line's release notes give its end-of-life date as 18 August 2026.[5] If you're on 4.14 or anything older, there is no patch coming. You have to move.

## THE BACKGROUND YOU NEED

### What Camel is, in two paragraphs

**Apache Camel** *≈ an open-source Java framework for connecting systems: it moves and transforms data between files, queues, databases, HTTP APIs and hundreds of other technologies* is an **integration framework** *≈ a library that provides ready-made building blocks for "take data from A, transform it, send it to B" so you don't hand-write the plumbing* . You describe the plumbing as a **route** *≈ a Camel pipeline: where messages come from, what happens to them, and where they go* , usually in a few lines of Java:

```
from("sftp://partner.example.com/outbox?delete=true")  // poll a server
    .unmarshal().csv()                                  // parse the file
    .to("kafka:orders");                                // publish each order
```

Each string like `sftp://...` names a **component** *≈ a Camel plugin for one technology (FTP, Kafka, HTTP, Redis…); Camel ships hundreds of them* and configures an **endpoint** *≈ one configured use of a component, written as a URI such as kafka:orders* . The endpoint at the start of a route is a **consumer** *≈ the endpoint that receives messages into a route (an HTTP server, a file poller, a queue listener)* ; endpoints you send to are **producers** *≈ endpoints that send messages out of a route (an HTTP client call, a file upload, a Kafka publish)* . Between them travels an **exchange** *≈ the message envelope Camel passes along a route: a body plus headers and properties* , whose **headers** *≈ key-value metadata attached to a message, like HTTP headers; Camel also uses special headers starting with "Camel" to steer its own behaviour* carry both business data and instructions for Camel itself.

That last point matters for everything below. Camel's own control headers (for example `CamelHttpUri` , which overrides where an HTTP producer sends a request) live in the same map as headers that arrived from the outside world. Keeping the two apart is the job of a **HeaderFilterStrategy** *≈ the Camel component that decides which headers may enter from, or leave to, the outside world* . You can run Camel inside Spring Boot (through the `camel-`

`spring-boot` starters), on Quarkus, or on its own. The current 4.23.0 release is built against Spring Boot 4.1.1 and Spring Framework 7.0.9, and supports JDK 17, 21 and 25.[6,7]

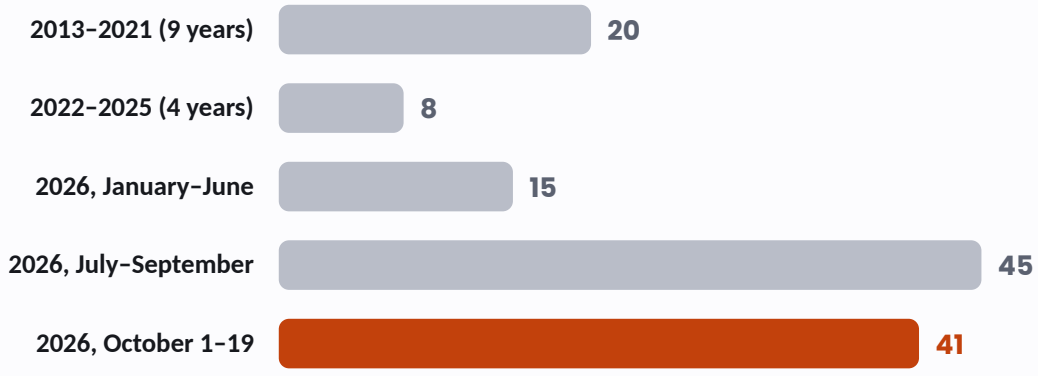
### Camel has been here before

In March 2025, Camel published CVE-2025-27636: its default incoming header filter "only blocks headers starting with 'Camel', 'camel', or 'org.apache.camel.'", so an attacker could send a header with different capitalisation and, for example, make the bean component "call another method on the bean, than was coded in the application", or make the JMS component send a message "to another queue".[8] It's the same pattern as several of this week's bugs: data from outside the route being treated as instructions to Camel.

### Why 2026 looks so different

Here's what the project's own advisory archive looks like when you count the advisory pages by publication date. We counted them in the Camel website's source repository on 20 October.[9]

**Apache Camel security advisories published, by period**




Our count of content/security/CVE-*.md pages in the apache/camel-website repository by their 'date' field (checked 20 Oct 2026). 2013–2025 total 28 pages; 2026 so far 101, of which 28 credit the Claude Security Scan Tool.

Before 2026, the archive grew by a handful of advisories a year. Since January it has grown by 101, and October alone (41 so far) outnumbers all 28 advisories from 2013 to 2025 combined.[9] Before October, the biggest single source was a human: twenty of this year's earlier advisories credit one researcher, Yu Bao from PayPal.[9] The October spike is the AI one.

That AI work comes from **Project Glasswing** *≈ an Anthropic initiative, announced 7 April 2026, that gives defenders access to an unreleased model to find and fix vulnerabilities in critical software*. Anthropic announced it on 7 April 2026 with partners including AWS, Apple, Google, Microsoft and the Linux Foundation, built around an unreleased model, Claude Mythos Preview, which Anthropic says it does "not plan to make … generally available". Its funding includes "$1.5M to the Apache Software Foundation".[10] In October, The Hacker News reported that Glasswing had verified at least 129,000 vulnerabilities between April and July, more than 33,000 of them rated critical or high. A VulnCheck analysis it cites found only 2 of 300 Anthropic- or Glasswing-credited vulnerabilities exploited in the wild.[11] We've watched this trend for weeks: Spring's flood of AI-assisted reports

(issue #2), IBM and Red Hat's 400 Java library bugs (#8), and Netty's 16 advisories without CVE numbers (#9). Camel's batch is the clearest example yet of what happens when a large Java codebase gets the full treatment.

> **A note on bias.** This newsletter is researched and written with the help of Claude, the same family of models credited in 28 of these advisories. We've stuck to what the advisories and the project say, and we'd write the same story whoever had found the bugs.

## HOW IT WORKS: SIX KINDS OF MISTAKE

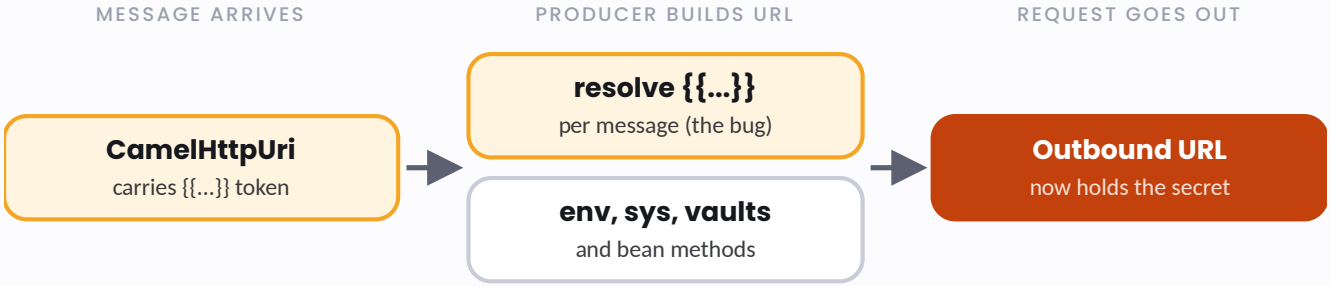
We read all 39 advisories. Most of them fall into six families, and the families are more useful to you than the CVE numbers, because they tell you what to look for in your own routes, Camel or not.

### 1. Message data treated as configuration or code

The most instructive bug in the batch is **CVE-2026-78164** (HIGH). Camel supports **property placeholders** ≈ *tokens in double curly braces, such as env:APP_HOME or sys:app.home, written in a route or config file and replaced by Camel with configuration values*. They're meant for route authors: you write `.to("http://localhost/{{basePath}}")`, and Camel fills in `basePath` from configuration when it builds the endpoint.[12,13]

The HTTP producers (in camel-http, camel-netty-http, camel-undertow and camel-vertx-http) also accept a message header, `CamelHttpUri` or `CamelRestHttpUri`, that overrides the target URL. And at seven places in the code, they ran that header's value through `resolvePropertyPlaceholders()` before using it.[14] So a `{{...}}` token arriving *inside a message* got the same treatment as one written by the route author. The advisory spells out what that reaches: "operating-system environment variables, JVM system properties, every PropertiesFunction registered on the classpath - including the AWS, Azure, GCP, HashiCorp, IBM, CyberArk and Kubernetes secret functions - and, through camel-bean, the invocation of a no-argument method on any bean in the registry."[14]

**CVE-2026-78164: how a header became a secrets lookup**




From the CVE-2026-78164 advisory. Fixed releases treat {{...}} in these two headers as literal text; placeholders in the route's own endpoint URI are still resolved once, at build time.

Three details make it worse than it first looks. The route author "opted into none of this: a plain static `to("http://backend/api")` was sufficient". Getting a value into the header "did not always require header injection, because several Camel HTTP consumers populate CamelHttpUri from the request line themselves as a documented reverse-proxy feature". And even with no outbound request, an unresolvable placeholder

produced an error message that echoed the URI, which "distinguishes a set from an unset configuration key".[14] The behaviour dates back to the old camel-http4 and camel-netty4 components, long before Camel 4.

**Closing CVE-2026-78164 in a route you can't upgrade yet**

**Exposed (Camel 4.x before the fix)**

```
from("netty-http:http://0.0.0.0:8080/api")
  .to("http://backend/api");

// If CamelHttpUri reaches the producer
// holding {{env:DB_PASSWORD}}, the token
// is expanded before the call goes out.
```

**Workaround from the advisory**

```
from("netty-http:http://0.0.0.0:8080/api")
  .removeHeader("CamelHttpUri")
  .removeHeader("CamelRestHttpUri")
  .to("http://backend/api");

// 4.23.0 / 4.22.1 / 4.18.5: {{...}} in
// these headers is plain text anyway.
```

Illustrative route. Workaround and fix behaviour from the CVE-2026-78164 advisory and the Camel 4.23 upgrade guide. bridgeEndpoint=true is NOT a workaround; skipControlHeaders=true works on camel-http only.

The same family includes **CVE-2026-83531** (HIGH) in the Dynamic Router component, whose control endpoint compiled a routing rule from "an expression and an expression language supplied by the control message". Because Camel's `simple` language "is always present", anyone who could send a control message could run an expression "with the privileges of the Camel process". The fix adds an `allowPredicateFromMessage` option that defaults to `false`.[15] And **CVE-2026-85312** (MEDIUM): the saga processor consulted a `Long-Running-Action` header "that sits outside the filtered Camel namespace", so an outside sender could choose which saga a message joined.[16]

### 2. Java deserialization without a filter

**Java deserialization** *≈ turning bytes back into Java objects with ObjectInputStream; dangerous on untrusted input because constructing objects can run code* is the oldest bug class in enterprise Java. The danger is a **gadget chain** *≈ a sequence of ordinary library classes whose deserialization callbacks, chained together, end up doing something harmful such as running a command*: if the classes are on your classpath, a crafted byte stream can trigger them during `readObject()`, before your code ever sees the object. The JDK's defence is an **ObjectInputFilter** *≈ a JDK filter (JEP 290) that accepts or rejects each class while a stream is being deserialized; set JVM-wide with -Djdk.serialFilter*.

Three advisories in this batch are about Camel components that deserialized network data with no filter, and in each case the deserialization happened *inside another library*, which is why earlier audits missed them. In **CVE-2026-78165** (HIGH), camel-spring-redis used Spring Data Redis's default `JdkSerializationRedisSerializer` on every message published to a subscribed channel, so "an actor able to issue PUBLISH on a subscribed channel reached the deserialization sink directly". The advisory notes: "The component never constructed a java.io.ObjectInputStream itself, which is why earlier passes over the Camel codebase for unfiltered Java deserialization did not reach it." The behaviour dates back to Camel 2.11.0.[17] **CVE-2026-78162** (MEDIUM) is an incomplete fix: an earlier advisory added a filter to four of the six places where camel-hazelcast creates a Hazelcast instance, and missed two.[18]

And **CVE-2026-88900**, Monday's addition, is the cleanest example of why *where* you filter matters. The JGroups consumer called `Message.getObject()`, and "JGroups Java-deserializes the payload through an ObjectInputStream on its own receive thread as the message arrives". With the default channel, the stock JGroups stack "multicasts over UDP without the AUTH protocol and without encryption", so any host that can reach the multicast group can join and send.[2]

**CVE-2026-88900: why a check inside Camel comes too late**




From the CVE-2026-88900 advisory. 4.22.1 and 4.18.5 add a post-read class check only; 4.23.0 also refuses to start a consumer unless a pre-read control (jdk.serialFilter, jgroups.deserialization.filter, or AUTH/encryption) is configured.

The advisory is unusually honest about this: the new class check "runs on the object that JGroups has already deserialized, so it narrows the body types that reach a route but cannot stop code that runs during deserialization". That's why Camel 4.23.0 makes the consumer "refuse to start" unless you configure a control that acts *before* the read. The escape hatch, `acceptAllObjects=true`, is "classified as an insecure serialization setting".[2] Because the guard breaks existing consumers on the default channel, it wasn't backported: on 4.22.1 and 4.18.5 you must set the JVM-wide filter yourself.[2]

> 🧪 **Try it: what a JVM-wide filter does**
>
> The advisories recommend this filter for camel-spring-redis: `-Djdk.serialFilter=!java.net.**;java.**;javax.**;org.apache.camel.**;!*` (reject `java.net`, allow core Java and Camel types, reject everything else).[17] We serialised a `HashMap` holding a `java.net.URL`, the class used by the well-known "URLDNS" probe, and read it back on OpenJDK 21.0.12.1. Without the flag, `readObject()` returned the map. With it, the read failed with `InvalidClassException: filter status: REJECTED`. The JDK's filter is in the `ObjectInputStream` itself, so it also covers streams that libraries such as Spring Data Redis create on your behalf.[19]

### 3. Trusting what the remote side tells you

Four HIGH advisories (CVE-2026-79983 to 79986) cover the FTP/SFTP, MINA SFTP, SMB and Azure Files consumers. They built each file's remote path by joining the polled directory with the name the server reported in its directory listing, with "no lexical normalization and no check that the result stayed inside the directory being polled". A listing entry containing `../` could make Camel read, delete or rename files elsewhere "using the endpoint's own credentials". On plain FTP, "an on-path attacker can rewrite a LIST response without holding any credentials". The `jailStartingDirectory` option, enabled by default, "claimed

a boundary that was not enforced here".[20] These four were found by Dilrevx from NSSL at Shanghai Jiao Tong University.[1]

Similar logic sits behind **CVE-2026-85251** (HIGH): with `followRedirects` enabled, camel-http re-attached the `Authorization` header to "whatever host the Location header named, so the remote server chose which host received the credentials". And **CVE-2026-80344** (HIGH) in the AS2 B2B component let the trading partner choose the address Camel would connect to for an asynchronous receipt, and with which credentials.[21,22] In security jargon, that's **SSRF** *≈ server-side request forgery: tricking a server into making network requests to a destination the attacker chooses*.

### 4. Security checks that fail open

A **fail-open** *≈ a security check that, when something goes wrong, lets the request through instead of blocking it* bug is the scariest kind, because the protection looks like it's working. In **CVE-2026-80312** (HIGH), camel-oauth's processors, on failing to authenticate, "fell through and returned control to the route, so the request continued to the protected endpoint as though authentication had succeeded".[23] In **CVE-2026-80343** (HIGH), camel-netty-http checked security constraints against the request path case-sensitively, while routing matched it case-insensitively, so "altering the case of a path segment skipped the constraint and still reached the endpoint".[24] Both were found by the Claude scanner. Two MEDIUM advisories cover camel-shiro and an `iss`-claim check in camel-keycloak.[1]

### 5. XML parsers with their guard down

Three advisories are **XXE** *≈ XML External Entity processing: an XML document declares an "entity" pointing at a file or URL, and a naive parser fetches it, leaking files or making requests*. The subtle one is **CVE-2026-85250** (HIGH): Camel's XPath support hardened its default parser, but `documentType=InputSource` handed the payload to the JDK's XPath engine, which "builds a DocumentBuilder of its own using the JDK defaults", so a deployment choosing that option "for streaming reasons had no indication it was also choosing a weaker parser".[25] The others affect camel-smooks (HIGH) and camel-fop (MEDIUM).[1]

### 6. Shared state under concurrency

Finally, bugs that only appear under load. **CVE-2026-85248** (MEDIUM) in camel-pqc shared "one java.security.Signature instance … across concurrent exchanges without synchronization, so interleaved use could make a verification succeed for data it never covered".[26] In **CVE-2026-85317**, camel-http's JVM-wide OAuth2 token cache "was keyed on fewer fields than shape the token", so one endpoint could receive a token minted for a different scope.[27] A thrift advisory describes "data from one exchange visible to the next".[1]

## WHAT IT MEANS FOR YOU

**Step 1: find out whether you run Camel at all**

Camel often arrives indirectly: through a Spring Boot starter, a Quarkus extension, or a vendor product. Run `mvn dependency:tree -Dincludes=org.apache.camel` or `gradle dependencies | grep org.apache.camel` on every service, and check container images with your SCA tool. Note the version of `camel-core` and the list of components, because most advisories only matter if you use that component.

**Step 2: upgrade to a fixed line**

Pick the row from the table at the top. If you're on an LTS line (4.18 or 4.22), the patch upgrade is the safest move. The Camel release notes put 4.18's end of life at 16 February 2027 and 4.22's at 10 August 2027.[5] If you're on 4.14 or 3.x, plan the bigger jump now. The 4.23 upgrade guide runs to more than 5,300 lines, and many of the fixes deliberately change defaults: routes that pass their own classes through Redis, Hazelcast or JGroups must widen the new deserialization allow-lists, and routes that ran steps *after* a failed OAuth check will now stop.[12]

**Step 3: if you can't upgrade this week**

The advisories each list a workaround. The most broadly useful ones:

- **Strip control headers at the edge.** Remove `CamelHttpUri` and `CamelRestHttpUri` before HTTP producers, and apply a strict inbound `HeaderFilterStrategy` on consumers exposed to untrusted senders.[14]
- **Set a JVM-wide serialization filter** on any JVM that consumes Redis, Hazelcast or JGroups data, and turn on authentication and encryption for those clusters.[17,2]
- **Prefer SFTP/FTPS over FTP**, restrict file-name patterns with `include` / `antInclude`, and avoid `delete=true` or `move` against servers you don't fully trust.[20]
- **Don't use** `documentType=InputSource` for XPath over untrusted XML.[25]

**Step 4: switch on Camel's own guard rails**

Camel has a startup check that flags insecure options, in four categories: plain-text secrets, weakened TLS, Java deserialization and development features. By default it only warns. Setting `camel.main.profile = prod` makes it fail: "the application refuses to start with any insecure configuration unless explicitly overridden".[28] On 4.23.0 (and, for the Dynamic Router option, 4.22.1), the new escape hatches `acceptAllObjects` and `allowPredicateFromMessage` are marked as insecure options, so the `prod` profile refuses them.[2,15]

### 💥 Watch out for

- **"Fixed" isn't the same as "safe" for deserialization on 4.22.1 and 4.18.5.** The JGroups class check there runs after deserialization. You still need `jdk.serialFilter` or an authenticated, encrypted channel.[2]
- **Incomplete fixes happen.** CVE-2026-78162 exists because an earlier Hazelcast fix (shipped in 4.14.8, 4.18.3 and 4.21.0) covered four of six code paths.[18] Re-check after each batch, not just once.
- **Unsupported lines get nothing.** The JGroups advisory lists 4.15–4.17 and 4.19–4.21 as end-of-life too.[2] Only 4.18, 4.22 and 4.23 are fixed.
- **Severity isn't exposure.** A HIGH in a component you don't use matters less than a MEDIUM in the HTTP consumer facing the internet. Triage by component.

## WHAT HAPPENS NEXT

Camel has now published advisories in two big batches this year: 33 in July and 41 so far in October.[9] With Glasswing's scanning continuing across ASF projects, expect more batches like this one, from Camel and from its neighbours. The practical lesson is boring and important: stay on a supported line, because the next 39 fixes will only land there.

### 📖 Words from this story

**Apache Camel**: an open-source Java framework for connecting systems by moving and transforming data between them.

**Integration framework**: a library of ready-made building blocks for "take data from A, transform it, send it to B".

**Route**: a Camel pipeline describing where messages come from, what happens to them, and where they go.

**Component**: a Camel plugin for one technology (FTP, Kafka, HTTP, Redis and so on).

**Endpoint**: one configured use of a component, written as a URI.

**Consumer**: the endpoint that receives messages into a route.

**Producers**: endpoints that send messages out of a route.

**Exchange**: the envelope Camel passes along a route: a body plus headers and properties.

**Headers**: key-value metadata on a message; Camel's own control headers start with "Camel".

**HeaderFilterStrategy**: the Camel component that decides which headers may cross the boundary with the outside world.

**Project Glasswing**: an Anthropic initiative (April 2026) that uses an unreleased model to find and fix vulnerabilities in critical software.

**Property placeholders**: tokens in double curly braces, such as `env:APP_HOME`, that Camel replaces with configuration values.

**Java deserialization**: turning bytes back into objects with `ObjectInputStream`, which can run code.

**Gadget chain**: library classes whose deserialization callbacks, chained together, do something harmful.

**ObjectInputFilter**: the JDK's per-class allow/deny filter for deserialization (JEP 290), set JVM-wide with `-Djdk.serialFilter`.

**SSRF**: server-side request forgery, making a server call a destination the attacker chooses.

**Fail-open**: a check that lets a request through when something goes wrong.

**XXE**: XML External Entity processing, where a parser fetches files or URLs named inside the document.

STORY 2

# Shenandoah stops degenerating

JDK 28 · HOTSPOT · SHENANDOAH GC · INTEGRATED 19 OCT 2026 · ~14 MIN READ

Every low-pause garbage collector makes a promise it can't always keep: "I'll clean up while your application runs." Sometimes the application allocates memory faster than the collector can free it, and something has to give. For seven years, Shenandoah's answer was to stop every application thread and finish the job in a hurry. On Monday, that answer was deleted from the JDK. In JDK 28, only the threads that actually need memory will wait, and the collector will throw more workers at the problem. It's a small change in what you'll see in your logs, and a big change in philosophy.

## WHAT HAPPENED

On 19 October 2026 at 18:37 UTC, William Kemper integrated **"8387790: Shenandoah: Remove degenerated cycles"** into the JDK main line, together with a related fix, "8338534: Shenandoah: Handle alloc failure differently when immediate garbage is available". It was reviewed by Kelvin Nilsen, Aleksey Shipilev and Roman Kennke, and touches 112 files, adding 1,721 lines and removing 3,079. The two files that implemented degenerated cycles, `shenandoahDegeneratedGC.cpp` and its header, are gone entirely (599 lines).[29]

The command-line switch that controlled the feature, `-XX:ShenandoahDegeneratedGC`, becomes an **obsolete flag** ≈ *a JVM option that is still accepted but ignored with a warning; one release later it is "expired" and the JVM refuses to start if you pass it* in JDK 28 and expires in JDK 29.[29] Because this is a change to the collector's internals rather than a new API, it didn't need a JEP. It's in the main line, so it will appear in the next JDK 28 **early-access build** ≈ *a weekly pre-release JDK binary for testing upcoming features* (the latest, build 20, was tagged on 15 October).[30] JDK 28 itself is a non-LTS release due in March 2027.[40]

The pull request's own summary of the motivation is blunt: handing off from concurrent to degenerated cycles "has caused many defects", and removing them "should simplify the code and lower maintenance costs".[31]

## THE BACKGROUND YOU NEED

### Shenandoah in one minute

**Shenandoah** ≈ *a low-pause garbage collector in OpenJDK that moves objects while your application keeps running* arrived as an experimental collector in JDK 12 via JEP 189, whose central promise is that "pause times with Shenandoah are independent of heap size".[32] Like G1, it divides the heap into **regions** ≈ *fixed-size chunks of the heap that the collector manages individually*. What makes it special is **concurrent evacuation** ≈ *copying live objects out of a region into a fresh one while application threads keep running and keep using those objects*: your threads (in GC jargon, **mutators** ≈ *the application threads, which keep changing the object graph while the collector works*) keep running during almost all of the work.

A normal Shenandoah cycle runs three concurrent phases, separated by very short **stop-the-world pauses** ≈ *moments when the JVM halts all application threads at a safepoint so the collector can do work that needs a consistent view*:

1. **Mark**: find which objects are still reachable.

2. **Evacuate**: copy live objects out of the most-garbage regions (the **collection set** ≈ *the group of regions the collector has chosen to empty and reclaim in this cycle*).
3. **Update references**: fix every pointer that still points at an old copy.

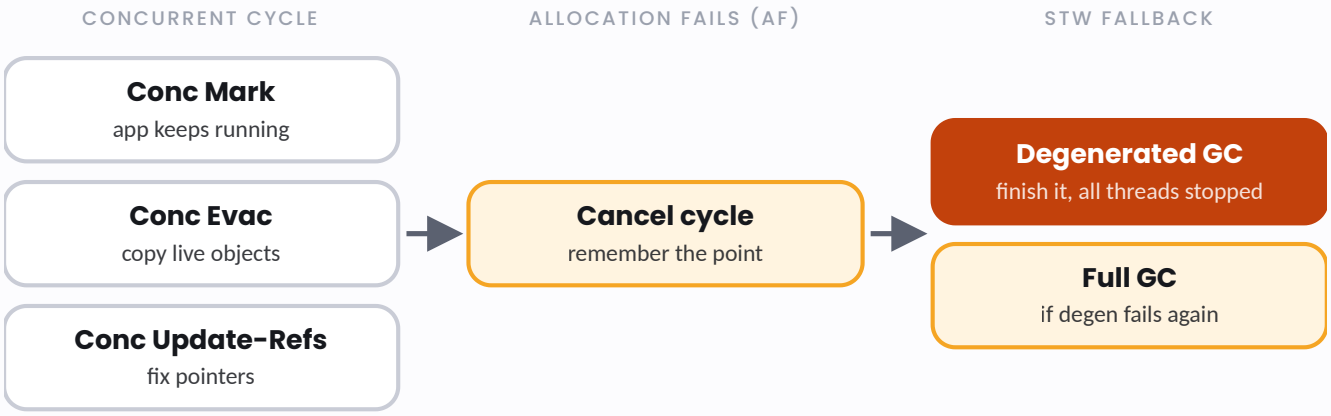
At the end, the emptied regions become free memory. As we explained in issue #11, generational Shenandoah (experimental in JDK 24 via JEP 404, a product feature since JDK 25) adds young and old generations to the same design, and JEP 535 proposes making it the default in JDK 28. In today's main line the default mode is still `satb`, the single-generation mode.[33,34]

### The problem: what if you run out mid-cycle?

A concurrent collector is in a race. It starts a cycle when it predicts it can finish before the heap fills up. If an allocation spike beats that prediction, a thread asks for memory and there is none. That's an **allocation failure** ≈ *the moment a thread asks the heap for memory and no free space is available*.

Shenandoah's answer used to come in three escalating steps. Until JDK 26 there was a **pacer** ≈ *a Shenandoah mechanism (removed in JDK 26) that slowed down allocating threads with tiny delays when the GC was falling behind*, which could stall allocating threads briefly; it was disabled and purged in July 2025.[35] Next came the **degenerated GC** ≈ *Shenandoah's fallback: stop all threads and finish the interrupted concurrent cycle in a stop-the-world pause, from wherever it had got to*. The cycle would be cancelled and then *continued*, stop-the-world, from the point it had reached. Last came a **Full GC** ≈ *a stop-the-world collection that marks and compacts the entire heap; the slowest and last-resort option*. The removed source code contained a lovely ASCII diagram of exactly this; here it is as a picture.

**Allocation failure before JDK 28: degrade, then escalate**



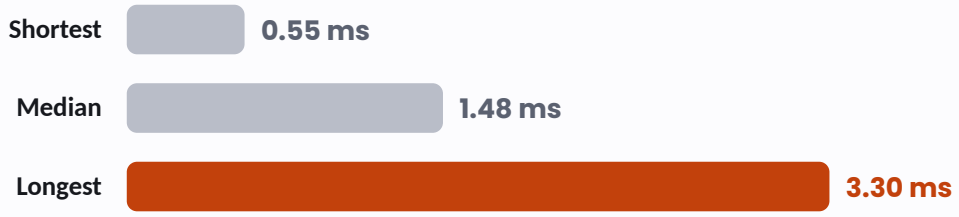

Redrawn from the comment in ShenandoahControlThread::service_concurrent_normal_cycle(), removed by openjdk/jdk commit 52e860f. Degenerated cycles used -XX:ParallelGCThreads workers (gceasy.io).

The degenerated cycle was a clever compromise. A full GC throws away everything the concurrent cycle had done; a degenerated cycle reuses it, so it's usually much shorter. GC-analysis firm gceasy describes the trade-off as degenerated pauses typically "<100 ms, but can be more, depending on the degeneration point", and full GC pauses as ">100 ms".[36] JEP 404's success metrics for generational mode were literally to "reduce or eliminate the number for allocation stalls and the need for full or degenerated collections".[33]

### We watched it happen on JDK 21

To see the old behaviour, we ran a small program on OpenJDK 21.0.12.1 with Shenandoah: a 256 MB heap with about 150 MB permanently live, churning out 4 to 12 KB arrays as fast as possible for eight seconds, on a 2-CPU sandbox with `-XX:ConcGCThreads=1`. It's deliberately cruel. The JVM's exit statistics reported **384 successful concurrent GCs and 68 degenerated GCs**, all "caused by allocation failure": 65 "happened at Outside of Cycle" (the heap ran out before a cycle had even started) and 3 "at Mark".[19]

**Degenerated GC pauses in our JDK 21 stress run (68 pauses)**




Our run: OpenJDK 21.0.12.1, -XX:+UseShenandoahGC -Xmx256m, 2 CPUs, from Shenandoah's -Xlog:gc+stats summary. A synthetic worst case on a tiny heap, not a benchmark. In an earlier run, one degenerated cycle escalated to a full GC and paused for 38 ms while freeing nothing (150M->150M).

Those pauses are short because the heap is tiny. The point is the shape: every time the collector lost the race, *every* thread in the JVM stopped. In our first attempt, the log also showed the escalation path: `Cancelling GC: Upgrade To Full GC`, then a 38 ms pause that went from 150 MB to 150 MB.[19]

## HOW IT WORKS

### The new rule: only the hungry thread waits

In JDK 28, when a thread's allocation fails, it now does something much simpler. The new comment in the control thread says it in one sentence: "If allocation failure (af) happens during any of the concurrent phases, the allocating thread will block until the concurrent cycle completes."[29] Everyone else keeps running. That waiting is called an **allocation stall** ≈ *a thread blocking inside an allocation until the garbage collector has freed memory for it*.

Reading the new `ShenandoahHeap::allocate_memory()` and `ShenandoahController`, the process has five steps, summarised in the diagram below.[29]

**Stall and signal.** The thread increments a per-cycle stall counter, records which phase the collector was in, tells the control thread it needs a cycle, and waits on a monitor.[29]

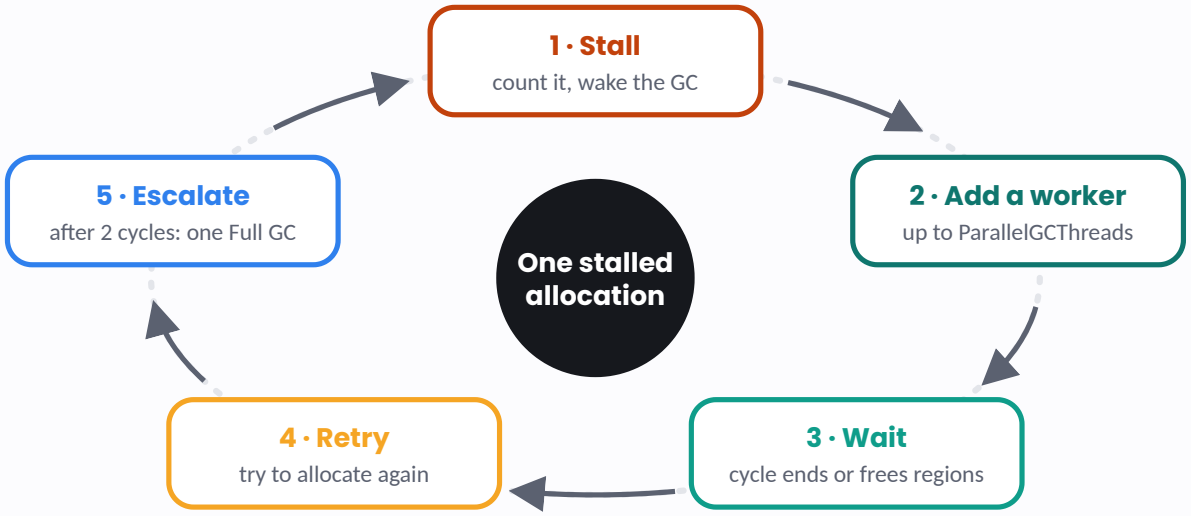
**Grow the workforce.** Each stall also nudges up the number of **concurrent GC workers** ≈ *the background threads that do marking, evacuation and reference updating while the application runs; sized by -XX:ConcGCThreads*, one at a time, up to `ParallelGCThreads`. At the end of a cycle with no stalls, the count steps back down by one, never below `ConcGCThreads`.[29] Before, the extra CPU arrived all at once inside a pause, because degenerated cycles ran with `ParallelGCThreads` workers.[36] Now the collector speeds up while your code keeps running. For now it

can only add workers between phases; a follow-up pull request (#32934) would let extra workers join mid-phase.[37]

**Wake up early if possible.** This is the second bug fix in the commit. At the end of marking, Shenandoah can often free whole regions that contain nothing live (so-called **immediate garbage** ≈ *regions found completely dead during marking, which can be freed at once without copying anything*). The new code wakes waiting threads right there, before evacuation starts, "so notify alloc waiters", instead of making them wait for the whole cycle.[29]

**Escalate, but reluctantly.** If a thread has waited through `ShenandoahFullGCThreshold` cycles (now 2, previously 3 degenerated cycles) and still can't allocate, and no full GC has run meanwhile, it requests exactly one full GC and retries once. If that fails too, the allocation fails for real, with an `OutOfMemoryError`.[29]

**What a thread does when its allocation fails (JDK 28)**




From ShenandoahHeap::allocate_memory() and ShenandoahController::handle_alloc_failure() at openjdk/jdk commit 52e860f. The '2' is the experimental flag ShenandoahFullGCThreshold.

### What about a failure while copying?

There's a second kind of allocation failure: a GC worker (or an application thread helping out) tries to copy an object during evacuation and finds no space for the copy. Here Shenandoah now relies on **self-forwarding** ≈ *marking an object that couldn't be copied as "forwarded to itself", so everyone treats it as already handled and it simply stays where it is*, which Roman Kennke introduced in April 2026 (JDK-8382636) and which already ships in JDK 27.[38,39] In JDK 28 the cycle doesn't stop when this happens. At the end, a collection-set region that still holds self-forwarded objects isn't thrown away; it's "partially recycled": the stragglers are un-forwarded, the gaps between them become filler, and the region's top is moved to just after the last surviving object, so "everything above this is reusable memory".[29]

**How Shenandoah learned to stop panicking**




Sources: JEP 189, JEP 404, openjdk/jdk commits 79f9d8d (Jul 2025), 02ad071 (Apr 2026, present at tag jdk-27-ga) and 52e860f (19 Oct 2026).

Seen this way, the change is the last step of a long simplification. Each of the old "graceful degradation" mechanisms has been replaced by something that keeps the application running.

### The argument against

Not everyone was immediately convinced. In the review, Roman Kennke, Shenandoah's original co-author, asked why degenerated cycles were no longer needed, since they existed to fail more smoothly under stress and protect latency, and asked for data showing no regression. Kemper's answer: it depends on when memory runs out, and stalling only the allocating thread "can beat a degenerated cycle that stops all threads". And, more fundamentally: "Even if there were a performance regression under stress, we would prefer it to instability under stress."[31] Kennke is listed among the commit's reviewers.[29]

## WHAT IT MEANS FOR YOU

### If you don't use Shenandoah

Nothing changes: G1 remains the default collector. Oracle's JDK builds don't include Shenandoah at all (as we noted in issue #11); you get it in OpenJDK builds such as Temurin, Corretto and Red Hat's.

### If you do: your pauses will look better, but measure latency

This is the important part. A degenerated GC was a **pause**: it showed up in GC logs, in pause-time metrics, and in JMX notifications. An allocation stall is **not** a pause. Only the stalled thread waits, and the JVM keeps running. So after you move to JDK 28, your "GC pause" dashboards may suddenly look great while some requests still wait just as long, or longer.

Here's what to adjust:

- **Watch per-thread stall logs.** JDK 28 logs `Allocation Stall (<thread name>)` when a stall begins (at `gc+start`) and `Allocation Stall (<thread name>) <n>ms` when it ends (at `gc` info level).[29]
- **Use JFR's event.** The allocating path still emits the `AllocationRequiringGC` event (Java Flight Recorder's `jdk.AllocationRequiringGC`), so you can see which threads hit the wall.[29]

- **Update anything that counts degenerated GCs.** The `"Degenerated GC"` action disappears from GC pause notifications (the JDK's own test no longer expects it), and the exit statistics now report "Stalls", broken down by phase, instead of "Degenerated GCs".[29]
- **Remove** `-XX:-ShenandoahDegeneratedGC` from your startup scripts. JDK 28 will ignore it with a warning; JDK 29 will refuse to start.[29]
- **Treat stalls as a sizing signal.** The new exit message is the same advice as always: "Increase heap size, tune GC heuristics, or lower allocation rate to avoid stalls and Full GC cycles."[29]

**What an allocation emergency looks like in -Xlog:gc**

JDK 21 (our run, real output)

```
Failed to allocate Shared, 1024K
Cancelling GC: Allocation Failure
GC(35) Pause Degenerated GC
  (Outside of Cycle)
  184M->151M(256M) 1.607ms
```

JDK 28 (format from the source)

```
Allocation Stall (http-nio-8080-exec-7)
...concurrent cycle continues...
Allocation Stall (http-nio-8080-exec-7)
  12.345ms
```

Left: our OpenJDK 21.0.12.1 log, wrapped and with timestamps removed. Right: message formats from ShenandoahAllocationStallLogger at commit 52e860f; the thread name and duration are illustrative. The start line is logged at gc+start level.

**💥 Watch out for**

- **This is JDK 28 only.** It's a large rewrite of collector internals, so don't expect it in JDK 21 or 25 updates. The next LTS that will include it is planned to be JDK 29 (September 2027).[40]
- **Stall time hides from pause metrics.** A request that waited 40 ms for memory shows as zero GC pause time. Track tail latency and the stall log.
- `ShenandoahFullGCThreshold` **changed meaning.** It used to count back-to-back degenerated GCs (default 3); it now counts concurrent cycles a thread will wait through (default 2). It's an experimental flag, but if you set it, re-check.[29]
- **Not yet in a downloadable build.** The newest early-access build (28+20) predates the commit.[30]

## WHAT HAPPENS NEXT

Two follow-ups are already visible. Pull request #32934 would let extra workers join a phase that is already running, which the author says became necessary because, with degenerated cycles gone, Shenandoah "can only increase concurrent workers between phases".[37] And Kelvin Nilsen suggested in the review that recycling evacuated regions earlier could give stalled threads memory sooner, as future work.[31] Meanwhile JEP 535 (generational mode by default) still has to be integrated. If both land, JDK 28 will have the most changed Shenandoah since it became a product feature, and it will be worth testing your latency-sensitive services on the first early-access build that has them.

### 📖 Words from this story

**Obsolete flag**: a JVM option accepted but ignored with a warning; one release later it's "expired" and rejected.

**Early-access build**: a weekly pre-release JDK binary.

**Shenandoah**: a low-pause OpenJDK collector that moves objects while the application runs.

**Regions**: fixed-size chunks of the heap that the collector manages individually.

**Concurrent evacuation**: copying live objects to new regions while application threads keep running.

**Mutators**: the application threads, which keep changing the object graph while the GC works.

**Stop-the-world pauses**: moments when all application threads are halted so the GC can work.

**Collection set**: the regions the collector has chosen to empty in this cycle.

**Allocation failure**: a thread asks for memory and none is free.

**Pacer**: a Shenandoah mechanism, removed in JDK 26, that slowed allocating threads when the GC fell behind.

**Degenerated GC**: the old fallback that finished an interrupted concurrent cycle with all threads stopped.

**Full GC**: a stop-the-world collection that marks and compacts the whole heap.

**Allocation stall**: a thread blocking in an allocation until the GC frees memory for it.

**Concurrent GC workers**: background threads doing concurrent GC work, sized by `-XX:ConcGCThreads`.

**Immediate garbage**: regions found entirely dead during marking, freed without copying.

**Self-forwarding**: marking an object that couldn't be copied as forwarded to itself, so it stays in place.

## On the radar

**Also happened, in one line each. Some will get a full deep dive in a future issue.**

- **Oracle's October Critical Patch Update lands today (20 October):** it's the first JDK 21 update Oracle offers under the OTN license instead of the free NFTC terms.[41,42]
- **JEPs 545 and 546 are now Targeted to JDK 28:** ZGC gets faster startup and adaptive heap sizing, joining structured concurrency (JEP 543) on the list.[4]
- **jlink gets a `--security-properties` plugin:** a JDK 28 commit lets you bake `java.security` overrides into a custom runtime image, next to June's `--cacerts` plugin for trimming the trusted-certificate store.[43]
- **`-XX:AOTMode=on` is deprecated in JDK 28:** the JVM now warns and points you to the equivalent `-XX:AOTMode=required`.[44]
- **Azul Payara 7.5.0 fixes two remote-code-execution flaws:** CVE-2026-2586 (via the Administration Console) and CVE-2026-2587 (server-side template rendering).[4]
- **Gatherers4j 0.15.0:** the library of ready-made Stream Gatherers adds moving and running statistics and weighted moving means, with some breaking changes.[4]
- **Google ADK for Java 1.12.0:** Google's agent toolkit adds workflow-graph types ( `Node` , `BaseNode` , `Start` , `Edge` ).[4]

1. **Apache Camel security advisories: 38 CVEs fixed in 4.18.5, 4.22.1 and 4.23.0**, Andrea Cosentino, Apache Camel blog, 16 Oct 2026 (read from the apache/camel-website repository).
2. **CVE-2026-88900: Camel-JGroups unsafe Java deserialization**, Apache Camel security advisory, 19 Oct 2026.
3. **apache/camel-website commit 2fff00a "add the security advisory for CVE-2026-88900 (camel-jgroups)"**, 19 Oct 2026.
4. **Java News Roundup: JEPs for JDK 28, Azul Payara, Gatherers4j, Google ADK, Yupiik, Apache Camel CVEs**, Michael Redlich, InfoQ, 19 Oct 2026.
5. Apache Camel release announcements and notes for **4.23.0** (15 Oct 2026), **4.18.5** (11 Oct 2026), **4.22.1** (17 Sep 2026), plus release-note pages for 4.14.9, 4.18.0 and 4.22.0 (end-of-life dates), read from the apache/camel-website repository.

6. **apache/camel parent/pom.xml at tag camel-4.23.0** (spring-boot-version 4.1.1, spring-version 7.0.9, jgroups-version 5.5.7.Final).
7. **Release 4.23.0**, Apache Camel release notes (supported JDKs 17, 21, 25).
8. **CVE-2025-27636: Camel Message Header Injection via Improper Filtering**, Apache Camel security advisory, 9 Mar 2025.
9. **apache/camel-website**, `content/security/` advisory pages; our count by each page's `date` and `credit` fields, 20 Oct 2026.
10. **Project Glasswing**, Anthropic, announced 7 Apr 2026 (partners, Claude Mythos Preview, $1.5M to the Apache Software Foundation).
11. **Anthropic Expands Claude Access for Vetted Cyber Teams as Glasswing Finds 129,000 Flaws**, The Hacker News, 7 Oct 2026.
12. **Apache Camel 4.x Upgrade Guide, "Upgrading Camel 4.22 to 4.23"**, apache/camel at tag camel-4.23.0 (5,313 lines).
13. **Properties component**, Apache Camel documentation ( `{{env:APP_HOME}}` , `{{sys:app.home}}` examples).
14. **CVE-2026-78164: property placeholders resolved on CamelHttpUri / CamelRestHttpUri**, Apache Camel security advisory, 15 Oct 2026.
15. **CVE-2026-83531: Camel-Dynamic-Router code execution through a control-message predicate**, Apache Camel security advisory, 15 Oct 2026.
16. **CVE-2026-85312: Camel-Core saga hijacking**, Apache Camel security advisory, 15 Oct 2026.
17. **CVE-2026-78165: Camel-Spring-Redis unfiltered Java deserialization**, Apache Camel security advisory, 15 Oct 2026.
18. **CVE-2026-78162: Camel-Hazelcast incomplete fix for CVE-2026-43865**, Apache Camel security advisory, 15 Oct 2026.
19. Our own runs on OpenJDK 21.0.12.1 (Ubuntu build) in a 2-CPU sandbox, 20 Oct 2026: a jdk.serialFilter test with a serialized HashMap containing java.net.URL, and a Shenandoah allocation stress test (-XX:+UseShenandoahGC -Xmx256m -Xms256m -XX:ParallelGCThreads=2 -XX:ConcGCThreads=1 -Xlog:gc,gc+stats).
20. **CVE-2026-79983: Camel-FTP path traversal through file names in the server directory listing**, Apache Camel security advisory, 15 Oct 2026.
21. **CVE-2026-85251: Camel-HTTP credential disclosure on redirects**, Apache Camel security advisory, 15 Oct 2026.
22. **CVE-2026-80344: Camel-AS2 SSRF and credential disclosure through the asynchronous MDN address**, Apache Camel security advisory, 15 Oct 2026.
23. **CVE-2026-80312: Camel-OAuth authentication bypass**, Apache Camel security advisory, 15 Oct 2026.
24. **CVE-2026-80343: Camel-Netty-HTTP security constraint bypass through path case**, Apache Camel security advisory, 15 Oct 2026.
25. **CVE-2026-85250: Camel-XPath XXE with documentType=InputSource**, Apache Camel security advisory, 15 Oct 2026.
26. **CVE-2026-85248: Camel-PQC signature verification bypass under concurrent use**, Apache Camel security advisory, 15 Oct 2026.
27. **CVE-2026-85317: Camel-HTTP OAuth2 token confusion in the shared token cache**, Apache Camel security advisory, 15 Oct 2026.
28. **Security Policy Enforcement**, Apache Camel user manual (apache/camel main, 19 Oct 2026).
29. **openjdk/jdk commit 52e860f "8387790: Shenandoah: Remove degenerated cycles / 8338534: Shenandoah: Handle alloc failure differently when immediate garbage is available"**, William Kemper, 19 Oct 2026 (read with git: shenandoahController, shenandoahControlThread, shenandoahHeap, shenandoahHeapRegion, shenandoahConcurrentGC, shenandoahCollectorPolicy, shenandoahUtils, shenandoah_globals, arguments.cpp, tests).
30. **openjdk/jdk tag jdk-28+20**, 15 Oct 2026 (checked with git ls-remote on 20 Oct).
31. **8387790: Shenandoah: Remove degenerated cycles, pull request #31797**, openjdk/jdk on GitHub (description and review discussion).
32. **JEP 189: Shenandoah: A Low-Pause-Time Garbage Collector (Experimental)**, OpenJDK (delivered in JDK 12).
33. **JEP 404: Generational Shenandoah (Experimental)**, OpenJDK (delivered in JDK 24); generational mode's move to product status is described in **JEP draft 8356990**.
34. **shenandoah_globals.hpp in openjdk/jdk master**, read with git on 20 Oct 2026 (ShenandoahGCMode default "satb").
35. **openjdk/jdk commit 79f9d8d "8350050: Shenandoah: Disable and purge allocation pacing support"**, Y. Srinivas Ramakrishna, 23 Jul 2025.
36. **Shenandoah GC Tuning**, gceasy.io blog.
37. **Shenandoah: Increase concurrent worker threads within a phase, pull request #32934**, openjdk/jdk on GitHub (open, blocked on #31797).
38. **openjdk/jdk commit 02ad071 "8382636: Shenandoah: Use self-forwarding to handle OOM during evacuation"**, Roman Kennke, 30 Apr 2026.
39. openjdk/jdk at tag **jdk-27-ga**: shenandoahHeap.cpp contains the self-forwarding code; shenandoah_globals.hpp still defines ShenandoahDegeneratedGC (read with git, 20 Oct 2026).
40. **Oracle Java SE Support Roadmap**, Oracle (JDK 28 non-LTS, March 2027; JDK 29 LTS, September 2027; 'subject to change').

41. **Critical Patch Updates, Security Alerts and Bulletins**, Oracle (next CPU 20 Oct 2026; CSPU 17 Nov 2026).

42. **JDK 21 approaches end of permissive license**, Oracle Java blog.

43. **openjdk/jdk commit 776ffc5 "8377819: Security properties jlink plugin"**, Sean Mullan, 19 Oct 2026; cacerts plugin in **commit c289cf5 "8377102: cacerts jlink plugin"**, 26 Jun 2026.

44. **openjdk/jdk commit eb356f8 "8391227: Deprecate -XX:AOTMode=on"**, Ioi Lam, 19 Oct 2026.